TRƯỜNG ĐẠI HỌC THỦ ĐÔ HÀ NỘI

KHOA TOÁN & CÔNG NGHỆ THÔNG TIN

****

BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

CHỦ ĐỀ:

XÂY DỰNG THUẬT TOÁN BĂM (HASHING) 64-BIT

Giảng viên hướng dẫn : Hoàng Thị Mai

Sinh viên thực hiện:

Nguyễn Hoàng Nam : 223001664

Hà Nội, 10/2025

MỤC LỤC

I. MÔ TẢ BÀI TOÁN 4
1.1. Bối cảnh và Động lực Nghiên cứu 4
1.2. Định nghĩa Bài toán 4
1.3. Phát biểu Bài toán Chính thức 4
1.4. Giải pháp Kỹ thuật được Lựa chọn 5
1.5. Phạm vi và Giới hạn của Nghiên cứu 6
II. INPUT VÀ OUTPUT 7
2.1. Mô hình Xử lý Dòng Dữ liệu (Stream Processing) 7
2.2. Đặc tả Đầu vào (Input Specification) 7
2.3. Đặc tả Đầu ra (Output Specification) 9
2.4. Bảng Tóm tắt Input/Output 10
III. VÍ DỤ MINH HỌA CHI TIẾT 11
3.1. Tham số Khởi tạo 11
3.2. Trace Từng Bước: Băm Chuỗi "ABC" 11
3.3. Bảng Tổng hợp Trace 13
3.4. Kiểm chứng Hiệu ứng Tuyết Lở: So sánh "ABC" và "ABD" 13
3.5. Kiểm chứng thêm: Tính Xác định 14
3.6. Kiểm chứng: Chuỗi Rỗng 14
IV. TỔNG QUAN VÀ LÝ DO CHỌN ĐỀ TÀI 15
4.1. Lịch sử Phát triển của Hàm Băm 15
4.2. Họ Thuật toán FNV - Nền tảng của Nghiên cứu này 15
4.3. Tại sao Hàm Băm là Công cụ Thiết yếu trong CNTT? 16
V. PHÂN TÍCH CHI TIẾT THUẬT TOÁN 19
5.1. Kiến trúc Tổng thể 19
5.2. Phân tích Các Hằng số Đặc biệt 19
5.3. Phân tích Ba Phép Toán Cốt lõi 21
5.4. Phân tích Độ Phức tạp 22
5.5. Tính chất Bảo mật: Phân tích Lý thuyết 23
VI. MÃ NGUỒN CÀI ĐẶT (C++) 24
6.1. Cài đặt Đầy đủ và Giải thích 24
6.2. Kết quả Chạy Test Cases 28
6.3. Hướng dẫn Biên dịch 28
6.4. Phân tích Từng Đoạn Code Quan trọng 29
VII. ĐÁNH GIÁ VÀ KẾT LUẬN 30

7.1. Kiểm chứng Hiệu ứng Tuyết Lở (Avalanche Effect) 30
7.2. So sánh Toàn diện với Các Phương pháp Khác.....31
7.3. Bảng Tổng hợp So sánh Các Phương pháp.....34
7.4. Hạn chế của Thuật toán.....34
7.5. Hướng Phát triển Tiếp theo.....35
7.6. Kết luận 35
TÀI LIỆU THAM KHẢO.....37

I. MÔ TẢ BÀI TOÁN

1.1. Bối cảnh và Động lực Nghiên cứu

Trong kỷ nguyên số hóa toàn cầu, dữ liệu trở thành tài sản quý giá nhất của cá nhân, tổ chức và quốc gia. Mỗi giây, hàng tỷ giao dịch tài chính, tin nhắn cá nhân, hồ sơ y tế và tài liệu mật được truyền qua các mạng lưới số. Câu hỏi đặt ra là: Làm sao để đảm bảo rằng dữ liệu nhận được chính xác là dữ liệu đã được gửi đi, không bị chỉnh sửa hay giả mạo?

Đây chính là bài toán cốt lõi mà ngành mật mã học (cryptography) và bảo mật thông tin (information security) đã dành hàng thập kỷ để giải quyết. Một trong những giải pháp thanh lịch và hiệu quả nhất được đề xuất chính là hàm băm mật mã (cryptographic hash function) - một công cụ toán học có khả năng tạo ra "dấu vân tay số" (digital fingerprint) duy nhất và không thể làm giả cho bất kỳ khối dữ liệu nào.

1.2. Định nghĩa Bài toán

Bài toán nghiên cứu này tập trung vào việc tự thiết kế và cài đặt một hàm băm 64-bit hoàn chỉnh, thay vì sử dụng các thư viện mật mã học có sẵn như MD5, SHA-1, SHA-256, hay bcrypt. Quyết định này xuất phát từ nhiều lý do học thuật và kỹ thuật quan trọng:

- Mục đích giáo dục: Hiểu sâu về cơ chế hoạt động bên trong của hàm băm, thay vì chỉ sử dụng như "hộp đen".
- Mục đích nghiên cứu: Khảo sát tính đúng đắn của các tính chất lý thuyết khi áp dụng vào thực tế.
- Mục đích ứng dụng: Tạo ra một công cụ linh hoạt, có thể tùy biến cho các hệ thống nhúng (embedded systems) hoặc môi trường có ràng buộc về tài nguyên.

1.3. Phát biểu Bài toán Chính thức

Cho trước: Một chuỗi dữ liệu đầu vào M có độ dài tùy ý n byte, với $n \in \mathrm{N}$.

Yêu cầu xây dựng: Một hàm $H: \{0; 1\}^* \rightarrow \{0; 1\}^{64}$ ánh xạ dữ liệu đầu vào có độ dài bất kỳ sang một giá trị băm (hash value) có độ dài cố định 64 bit, sao cho hàm H thỏa mãn các tính chất bảo mật sau:

Tính chất 1: Tính Xác định (Determinism)

$$\forall M1 = M2 \Rightarrow H(M_1) = H(M_2)$$

Với cùng một đầu vào, hàm băm luôn luôn trả về cùng một kết quả, bất kể môi trường thực thi, thời gian hay bất kỳ yếu tố ngoại vi nào. Đây là điều kiện tiên quyết để hàm băm có thể được dùng cho mục đích xác minh.

Tính chất 2: Tính Một Chiều (One-Way / Pre-image Resistance)

$$Cho\ h = H(M)\ không\ thể\ tính\ ngược\ M\ từ\ \ h\ trong\ thời\ gian\ đa\ thức$$

Cho dù biết giá trị băm đầu ra h, kẻ tấn công không thể tìm lại được thông điệp gốc M trong thời gian thực tế. Tính chất này là nền tảng để hàm băm được dùng lưu trữ mật khẩu an toàn.

Tính chất 3: Hiệu ứng Tuyết Lở (Avalanche Effect)

$$Nếu\ M_1\ và\ M_2\ Khác\ nhau\ 1\ bit \rightarrow H(M_1)\ và\ H(M_2)\ \ khác\ nhau \approx 50\%\ số\ bit$$

Bất kỳ thay đổi nhỏ nào ở đầu vào - dù chỉ là một bit - phải dẫn đến sự thay đổi hoàn toàn không thể đoán trước ở đầu ra. Điều này đảm bảo rằng kẻ tấn công không thể suy luận về mối quan hệ giữa các đầu vào dựa trên sự tương đồng của đầu ra.

Tính chất 4: Kháng Va Chạm (Collision Resistance)

$$Rất\ khó\ tìm\ M_1 \neq M_2\ sao\ cho\ H(M_1) = H(M_2)$$

Va chạm (collision) là trường hợp hai thông điệp khác nhau tạo ra cùng một giá trị băm. Hàm băm tốt phải đảm bảo xác suất va chạm cực kỳ thấp.

1.4. Giải pháp Kỹ thuật được Lựa chọn

Sau khi khảo sát các kiến trúc hàm băm phi mật mã (non-cryptographic hash function) phổ biến, nghiên cứu này lựa chọn kiến trúc FNV-1a (Fowler–Noll–Vo) làm nền tảng, kết hợp với kỹ thuật xoay bit (bitwise rotation) để tăng cường hiệu ứng khuếch tán (diffusion).

Lý do lựa chọn FNV-1a:

- Thuật toán đơn giản, hiệu quả, phù hợp để phân tích từng thành phần.
- Có nền tảng lý thuyết rõ ràng về lý do các hằng số được chọn.
- Dễ dàng mở rộng và tùy biến thêm các bước xử lý.

1.5. Phạm vi và Giới hạn của Nghiên cứu

Nghiên cứu này không hướng đến việc tạo ra một thuật toán thay thế cho các chuẩn mật mã quốc tế như SHA-256 hay SHA-3. Thay vào đó, mục tiêu là:

- Minh họa cơ chế hoạt động của hàm băm thông qua một cài đặt cụ thể.
- Phân tích và kiểm chứng các tính chất lý thuyết trong thực nghiệm.
- So sánh cách tiếp cận này với các phương pháp bảo vệ dữ liệu khác.

Về giới hạn bảo mật: Thuật toán 64-bit trong nghiên cứu này không nên được dùng cho các ứng dụng bảo mật cấp cao trong thực tế, do không gian băm 64-bit (2^{64} giá trị) nhỏ hơn nhiều so với chuẩn tối thiểu 256-bit của các hàm băm mật mã hiện đại.

II. INPUT VÀ OUTPUT

2.1. Mô hình Xử lý Dòng Dữ liệu (Stream Processing)

Thuật toán băm được thiết kế theo mô hình xử lý dòng (stream processing) - một trong những mô hình tính toán căn bản và hiệu quả nhất trong khoa học máy tính. Thay vì nạp toàn bộ dữ liệu vào bộ nhớ rồi xử lý một lần, hệ thống đọc và xử lý từng byte một theo tuần tự, duy trì một trạng thái nội tại (internal state) duy nhất - chính là biến hashed - liên tục được cập nhật.

```
Dữ liệu đầu vào (chuỗi ký tự)
    |
    ▼
[Byte 1] → [Byte 2] → ... → [Byte n]   ← Luồng xử lý tuần tự
    |                           |
    ▼                           ▼
Trạng thái ban đầu        Trạng thái cuối
(DEFAULT value)           (Giá trị băm)
```

Ưu điểm của mô hình stream:

- Hiệu quả bộ nhớ: Chỉ cần $O(1)$ bộ nhớ phụ, bất kể kích thước đầu vào.
- Xử lý dữ liệu lớn: Có thể băm file gigabyte mà không cần nạp toàn bộ vào RAM.
- Tính linh hoạt: Dễ dàng tích hợp vào pipeline xử lý dữ liệu.

2.2. Đặc tả Đầu vào (Input Specification)

2.2.1. Kiểu dữ liệu và Kích thước

Thuộc tính	**Giá trị**
Kiểu dữ liệu	Chuỗi ký tự (String)
Bộ ký tự	UTF-8 / ASCII (mọi ký tự có mã byte từ 0–255)
Độ dài tối thiểu	0 byte (chuỗi rỗng)
Độ dài tối đa	Không giới hạn về lý thuyết

Thuộc tính	Giá trị
Đơn vị xử lý	1 byte (8 bit) tại một thời điểm

2.2.2. Xử lý Byte trong C++

Trong cài đặt C++, mỗi ký tự trong chuỗi đầu vào được ép kiểu về uint8_t (số nguyên không dấu 8-bit) trước khi xử lý:

$$u08\ BYTE = static_cast(c);$$

Bước ép kiểu này quan trọng vì:

- char trong C++ có thể là signed (từ -128 đến 127) trên một số nền tảng.
- Khi XOR với giá trị 64-bit, char âm sẽ bị sign-extend (mở rộng dấu) thành giá trị 64-bit âm, gây ra hành vi không mong muốn.
- uint8_t đảm bảo giá trị luôn trong khoảng $[0,255]$, cho phép XOR an toàn.

2.2.3. Các loại Đầu vào Thực tế

Thuật toán băm có thể xử lý mọi loại dữ liệu có thể biểu diễn dưới dạng chuỗi byte:

Loại dữ liệu	Ví dụ ứng dụng
Mật khẩu người dùng	Lưu trữ an toàn trong CSDL
Nội dung văn bản	Kiểm tra toàn vẹn tài liệu
Khóa từ điển (hash map)	Tăng tốc tra cứu dữ liệu
Nội dung file	Phát hiện file trùng lặp
Địa chỉ mạng	Phân phối tải (load balancing)
Token xác thực	Phát hiện giả mạo session

2.3. Đặc tả Đầu ra (Output Specification)

2.3.1. Dạng số nguyên 64-bit

Kết quả trực tiếp của hàm hashing() là một số nguyên không dấu 64-bit (uint64_t), với không gian giá trị:

$$H \in [0, 2^{64} - 1] = [0,18.446.744.073.709.551.615]$$

Đây là một không gian rất lớn - hơn 18 tỷ tỷ giá trị - đủ để đảm bảo xác suất va chạm ngẫu nhiên là cực kỳ thấp cho hầu hết ứng dụng thực tế.

2.3.2. Biểu diễn Hexadecimal

Số nguyên 64-bit được chuyển đổi sang chuỗi hexadecimal (hệ cơ số 16) để hiển thị. Kết quả là một chuỗi có độ dài cố định 16 ký tự (mỗi ký tự hex đại diện cho 4 bit, 16 × 4 = 64 bit).

```
Số nguyên 64-bit:  17116010699969094176
Chuỗi Hex:         ed88490c0f716220
← 16 ký tự hex, mỗi ký tự = 4 bit →
```

Các ký tự hex hợp lệ: 0–9 và a–f, tổng cộng 16 ký hiệu.

2.3.3. Zero-Padding và Tính Nhất quán

Hàm intToHex() sử dụng setfill('0') << setw(16) để đảm bảo kết quả luôn có đúng 16 ký tự, kể cả khi giá trị số nhỏ:

ss << setfill('0') << setw(16) << hex << hashed;

Tính nhất quán về độ dài đầu ra là yêu cầu bắt buộc: nếu độ dài thay đổi theo giá trị đầu ra, kẻ tấn công có thể khai thác thông tin đó.

2.4. Bảng Tóm tắt Input/Output

Thuộc tính	**Input**	**Output**
Kiểu dữ liệu	std::string	std::string (hex)
Độ dài	Tùy ý (≥ 0 byte)	Cố định (16 ký tự)
Biểu diễn	Ký tự ASCII/UTF-8	Hexadecimal
Không gian	Vô hạn về lý thuyết	2^64 giá trị
Ví dụ	"ABC"	"ed88490c0f716220"

III. VÍ DỤ MINH HỌA CHI TIẾT

3.1. Tham số Khởi tạo

Trước khi bắt đầu xử lý, thuật toán khởi tạo các hằng số sau:

Hằng số	Giá trị thập phân	Giá trị hex	Vai trò
DEFAULT	14.695.981.039.346.656.037	0xcbf29ce484222325	Trạng thái băm ban đầu
PRIME	1.099.511.628.211	0x100000001b3	Nhân tử khuếch tán
SMALL	13	0xD	Số bit xoay

Khởi tạo:

$$HASH = DEFAULT = 14695981039346656037\ (0xcbf29ce484222325)$$

3.2. Trace Từng Bước: Băm Chuỗi "ABC"

Chuỗi "ABC" gồm 3 ký tự, mỗi ký tự được xử lý qua 3 phép toán:

XOR → Nhân → Xoay.

Bước 1: Xử lý ký tự 'A' (ASCII = 65 = 0x41)

Phép toán 1 - XOR (Mixing)

```
HASH hiện tại = 14695981039346656037
              = 1100 1011 1111 0010 1001 1100 1110 0100
                1000 0100 0010 0010 0010 0011 0010 0101 (nhị phân)
  XOR với 65  = 0100 0001
  Kết quả     = 14695981039346656100
              = 1100 1011 1111 0010 1001 1100 1110 0100
                1000 0100 0010 0010 0010 0011 0110 0100
Giá trị sau XOR:  14.695.981.039.346.656.100
```

Phép toán 2 - Nhân với PRIME (Diffusion):

14695981039346656100 × 1099511628211 (mod 2^64)
= 12638222384927744748
Giá trị sau nhân: 12.638.222.384.927.744.748
Lưu ý: Phép nhân thực hiện theo modular arithmetic với modulo 2^64, nghĩa là chỉ lấy 64 bit thấp nhất của kết quả.

Phép toán 3 - Xoay trái 13 bit (Scrambling):

Giá trị = 12638222384927744748
= 0xAF5E2A8A2C0F95AC (hex)
Binary (64 bit):
1010 1111 0101 1110 0010 1010 1000 1010
0010 1100 0000 1111 1001 0101 1010 1100
Xoay trái 13 bit:
→ Kết quả = 9190035670081312236
= 0x7F829420BE32AD4C (hex)
Kết thúc bước 1: HASH = 9.190.035.670.081.312.236

Bước 2: Xử lý ký tự 'B' (ASCII = 66 = 0x42)

Lấy giá trị HASH từ bước 1 = 9.190.035.670.081.312.236 để tiếp tục:

XOR với 66:
9190035670081312236 XOR 66 = 9190035670081312174
Nhân với PRIME:
9190035670081312174 × 1099511628211 (mod 2^64)
= 16602153571942764554
Xoay trái 13 bit:
16602153571942764554 → 15318577832776778378
Kết thúc bước 2: HASH = 15.318.577.832.776.778.378

Bước 3: Xử lý ký tự 'C' (ASCII = 67 = 0x43)

XOR với 67:
15318577832776778378 XOR 67 = 15318577832776778313
Nhân với PRIME:

15318577832776778313 × 1099511628211 (mod 2^64)
Xoay trái 13 bit: → 17116010699969094176
Kết thúc bước 3: HASH = 17.116.010.699.969.094.176

Kết quả Cuối cùng

Input: "ABC"
Số: 17116010699969094176
Hex: ed88490c0f716220

3.3. Bảng Tổng hợp Trace

Bước	Ký tự	ASCII	Sau XOR	Sau Nhân	Sau Xoay
Init	-	-	-	-	14695981039346656037
1	'A'	65	14695981039346656100	12638222384927744748	9190035670081312236
2	'B'	66	9190035670081312174	16602153571942764554	15318577832776778378
3	'C'	67	15318577832776778313	(intermediate)	17116010699969094176

3.4. Kiểm chứng Hiệu ứng Tuyết Lở: So sánh "ABC" và "ABD"

Chỉ thay đổi ký tự cuối từ 'C' (67) sang 'D' (68) - thay đổi 1 bit cuối trong đầu vào:

	"ABC"	**"ABD"**
Đầu vào	41 42 43 (hex)	41 42 44 (hex)
Hash (hex)	ed88490c0f716220	ee28490c10814220

	"ABC"	"ABD"
Hash (số)	17116010699969094176	17155497571316719136

Phân tích bit-level của hai kết quả:

ABC: 1110 1101 1000 1000 0100 1001 0000 1100 0000 1111 0111 0001 0110 0010 0010 0000
ABD: 1110 1110 0010 1000 0100 1001 0000 1100 0001 0000 1000 0001 0100 0010 0010 0000

Đếm số bit khác nhau: ~16/64 bit (khoảng 25%), chứng tỏ hiệu ứng tuyết lở rõ ràng. Kết quả hoàn toàn khác nhau về mặt cấu trúc hexadecimal dù đầu vào chỉ khác một ký tự.

3.5. Kiểm chứng thêm: Tính Xác định

H("ABC") = ed88490c0f716220 (lần 1)
H("ABC") = ed88490c0f716220 (lần 2)
H("ABC") = ed88490c0f716220 (lần n)

Bất kể gọi bao nhiêu lần, kết quả luôn nhất quán - đây là điều kiện bắt buộc để thuật toán có thể dùng để xác minh dữ liệu.

3.6. Kiểm chứng: Chuỗi Rỗng

Input: "" (chuỗi rỗng, 0 ký tự)
Output: cbf29ce484222325

Khi không có byte nào được xử lý, giá trị băm chính là DEFAULT (hằng số khởi tạo), được biểu diễn thành chuỗi hex 16 ký tự.

IV. TỔNG QUAN VÀ LÝ DO CHỌN ĐỀ TÀI

4.1. Lịch sử Phát triển của Hàm Băm

Hàm băm không phải là phát minh của thời đại Internet. Lịch sử phát triển của chúng gắn liền với sự tiến hóa của khoa học máy tính và nhu cầu bảo mật thông tin qua nhiều thập kỷ.

Thập niên 1950–1960: Khởi đầu trong Khoa học Máy tính

Khái niệm hàm băm xuất hiện lần đầu tiên vào năm 1953, do Hans Peter Luhn tại IBM đề xuất, ban đầu phục vụ mục đích tra cứu và lập chỉ mục dữ liệu nhanh trong các hệ thống máy tính còn sơ khai. Lúc này, "hàm băm" chỉ đơn giản là ánh xạ từ khóa sang vị trí lưu trữ, hoàn toàn không có yếu tố bảo mật.

Thập niên 1970–1980: Nền tảng Mật mã học

Cùng với sự ra đời của mật mã học khóa công khai (Diffie-Hellman, 1976; RSA, 1977), nhu cầu về hàm băm mật mã học (cryptographic hash function) bắt đầu hình thành rõ ràng. Ralph Merkle (1979) đặt nền tảng lý thuyết cho hàm băm mật mã với cấu trúc Merkle-Damgård - vẫn là nền tảng cho nhiều thuật toán hiện đại.

Thập niên 1990: Kỷ nguyên MD và SHA

MD4 (Ron Rivest, 1990): Hàm băm 128-bit đầu tiên được sử dụng rộng rãi.

MD5 (Ron Rivest, 1992): Cải tiến MD4, trở thành chuẩn phổ biến nhất trong thập kỷ.

SHA-1 (NIST, 1995): Chuẩn liên bang Mỹ, 160-bit, được dùng trong SSL/TLS, Git.

FNV (Fowler-Noll-Vo, 1991): Ra đời từ dự án nội bộ tại NIST, tối ưu cho tốc độ.

Thập niên 2000–nay: Kỷ nguyên SHA-2 và SHA-3

SHA-256 / SHA-512 (NIST, 2001): Chuẩn mới sau khi MD5 và SHA-1 bị phá vỡ.

SHA-3 (Keccak) (NIST, 2012): Kiến trúc sponge hoàn toàn khác, chống lại các tấn công chiều dài mở rộng (length extension attack).

BLAKE3 (2020): Thuật toán băm song song hiện đại, nhanh hơn SHA-256 nhiều lần.

4.2. Họ Thuật toán FNV - Nền tảng của Nghiên cứu này

FNV (Fowler–Noll–Vo) là họ hàm băm phi mật mã (non-cryptographic) được phát triển bởi Glenn Fowler, Landon Curt Noll và Phong Vo vào năm 1991. Mặc dù không đạt mức bảo mật của SHA-256, FNV được đánh giá cao vì:

- Tốc độ cực nhanh: Chỉ dùng 3 phép toán (XOR, nhân, modulo) cho mỗi byte.

- Phân phối đều: Các giá trị băm được phân bố đồng đều trong không gian 64-bit.
- Đơn giản để cài đặt: Toàn bộ thuật toán có thể viết trong vài dòng code.

Hai biến thể chính:

- FNV-1: Nhân trước, XOR sau → hash = (hash * PRIME) XOR byte
- FNV-1a: XOR trước, nhân sau → hash = (hash XOR byte) * PRIME (được dùng trong nghiên cứu này)

Lý do FNV-1a tốt hơn FNV-1: Thứ tự XOR trước giúp mỗi byte có ảnh hưởng ngay lập tức đến toàn bộ kết quả nhân, tăng hiệu ứng khuếch tán (diffusion) mạnh hơn.

4.3. Tại sao Hàm Băm là Công cụ Thiết yếu trong CNTT?

Hàm băm ngày nay xuất hiện trong hầu hết mọi hệ thống phần mềm, đôi khi ẩn đi sau những lớp abstraction:

4.3.1. Xác thực Mật khẩu

Không một hệ thống bảo mật hiện đại nào lưu trữ mật khẩu dạng plaintext. Thay vào đó:

- Khi đăng ký: password → H(password + salt) → lưu vào CSDL
- Khi đăng nhập: input → H(input + salt) → so sánh với CSDL

Ngay cả khi CSDL bị đánh cắp, kẻ tấn công cũng không thể lấy lại mật khẩu gốc.

4.3.2. Kiểm tra Toàn vẹn Dữ liệu (Data Integrity)

Khi tải phần mềm từ Internet, nhà cung cấp thường kèm theo checksum (SHA-256):

- File gốc: firefox-setup.exe → SHA256 → a3f8c2...
- File bạn tải: firefox-setup.exe → SHA256 → a3f8c2... (khớp → an toàn)
- File bị nhiễm: firefox-setup.exe → SHA256 → 7b2d9e... (khác → cảnh báo)

4.3.3. Hệ thống Kiểm soát Phiên bản (Git)

Git sử dụng SHA-1 (và đang chuyển sang SHA-256) để định danh mọi commit, file và tree:

```
git log –oneline
a3f8c2e  Add new feature
7b2d9ef  Fix critical bug
```

Mỗi mã commit là một hash của toàn bộ trạng thái dự án tại thời điểm đó.

4.3.4. Cấu trúc Dữ liệu Hash Map / Hash Table

Trong lập trình, hash table là cấu trúc dữ liệu cho phép tra cứu $O(1)$ trung bình:

key → H(key) % table_size → index → value

4.3.5. Blockchain và Tiền mã hóa

Bitcoin sử dụng SHA-256 kép để đảm bảo tính bất biến của sổ cái phân tán - mỗi block chứa hash của block trước, tạo thành chuỗi không thể giả mạo.

4.4. Động lực Chọn Kiến trúc FNV-1a + Xoay Bit

Nghiên cứu này chọn FNV-1a làm nền tảng và bổ sung bước xoay bit (rotate left) vì các lý do sau:

Lý do giáo dục:

- FNV-1a đủ đơn giản để trace từng bước thủ công, giúp hiểu rõ tác động của từng phép toán.
- Việc bổ sung rotate left là một cải tiến có thể kiểm chứng được tác động lên phân phối và avalanche effect.

Lý do kỹ thuật:

- Xoay bit (rotate) khác với dịch bit (shift): xoay không mất thông tin (bit "tràn ra" quay lại phía kia), trong khi shift mất các bit bị đẩy ra ngoài.
- Xoay 13 bit (số nguyên tố) đảm bảo chu kỳ xoay dài hơn, tránh các pattern lặp.

So sánh FNV-1a thuần và FNV-1a + Rotate:

Tính chất	**FNV-1a thuần**	**FNV-1a + Rotate 13**
Tốc độ	Rất nhanh	Nhanh (thêm 1 phép toán/byte)
Avalanche Effect	Trung bình	Tốt hơn
Phân phối	Tốt	Tốt hơn
Độ phức tạp cài đặt	Rất thấp	Thấp

V. PHÂN TÍCH CHI TIẾT THUẬT TOÁN

5.1. Kiến trúc Tổng thể

Thuật toán được tổ chức theo mô hình lặp tuần tự (iterative sequential). Mỗi vòng lặp xử lý một byte đầu vào và cập nhật trạng thái 64-bit duy nhất. Hàm tổng thể có thể được biểu diễn toán học như sau:

$$H_0 = DEFAULT$$
$$Hi = RotLeft_{13}(H_{i-1} \oplus b_1) \times Prime, \quad i = 1, 2, \ldots, n$$
$$H(M) = H_n$$

Trong đó:

- b_i là byte thứ i của thông điệp M,
- $\oplus$ là phép XOR bitwise,
- $\times$ là phép nhân modulo 2^{64}.

5.2. Phân tích Các Hằng số Đặc biệt

5.2.1. Hằng số DEFAULT = 0xcbf29ce484222325

Giá trị mở đầu (offset basis) của FNV-1a là hằng số 14.695.981.039.346.656.037. Đây không phải số ngẫu nhiên - nó được chọn thông qua một quá trình khởi tạo đặc biệt:

Giá trị này là kết quả của việc băm chuỗi hạt giống (seed string) "chongo <Landon Curt Noll>" qua thuật toán FNV với offset ban đầu là 0. Điều này đảm bảo giá trị khởi đầu có độ entropy cao - các bit được phân bố đều, không có pattern rõ ràng - giúp ngay từ byte đầu tiên, thuật toán đã bắt đầu từ trạng thái "hỗn loạn" thay vì từ 0.

Biểu diễn nhị phân của DEFAULT:
1100 1011 1111 0010 1001 1100 1110 0100
1000 0100 0010 0010 0010 0011 0010 0101

Phân tích: ~50% bit là 1 (32/64 bit), phân phối gần như lý tưởng.

Tại sao không dùng DEFAULT = 0? Nếu DEFAULT = 0, hàm băm sẽ không hoạt động đúng vì 0 XOR b = b và 0 * PRIME = 0 - kết quả sẽ phụ thuộc hoàn toàn vào byte đầu tiên và bỏ qua toàn bộ trạng thái tích lũy.

5.2.2. Số nguyên tố PRIME = 0x100000001b3

Số nguyên tố FNV 1.099.511.628.211 được chọn với các tính chất đặc biệt:

Tính chất 1 - Cấu trúc thưa (sparse):

0x100000001b3 trong nhị phân:
0001 0000 0000 0000 0000 0000 0000 0000 0001 1011 0011

Đây là số nguyên tố có rất ít bit bằng 1 (chỉ 8/40 bit). Điều này tối ưu hóa phép nhân trên CPU: phép nhân với số có ít bit 1 tương đương với một vài phép dịch bit và cộng - nhanh hơn nhân tổng quát.

Tính chất 2 - Đảm bảo khuếch tán đầy đủ:

Khi nhân với số nguyên tố 64-bit, thay đổi ở các bit thấp được "khuếch tán" (diffuse) lên toàn bộ 64 bit. Cụ thể, nếu chỉ thay đổi 1 bit trong giá trị hiện tại, sau phép nhân với PRIME, khoảng 32 bit trong kết quả sẽ thay đổi - đây chính là diffusion effect mong muốn.

Tính chất 3 - Số nguyên tố đảm bảo tính toàn ánh:

Vì PRIME là số nguyên tố và không gian là $Z_{2^{64}}$, hàm $f(x) = x \times \text{PRIME} \pmod{2^{64}}$ là toàn ánh (bijection) - mỗi giá trị đầu vào ánh xạ đến một giá trị đầu ra duy nhất, đảm bảo không có thông tin nào bị mất.

5.2.3. Hằng số Xoay SMALL = 13

Con số 13 được chọn vì các lý do sau:

Lý do 1 - Số nguyên tố:

13 là số nguyên tố, đảm bảo khi xoay liên tục 13 bit, cần đúng 64/gcd(64,13) = 64 lần mới quay về vị trí ban đầu (vì gcd(64,13) = 1). Điều này tránh các chu kỳ ngắn.

Lý do 2 - Gần 1/5 chiều dài:

13/64 ≈ 0.20, nghĩa là xoay khoảng 1/5 chiều dài word. Thực nghiệm cho thấy giá trị xoay trong khoảng [11, 21] thường cho phân phối tốt nhất cho các word 64-bit.

Lý do 3 - Bù đắp cho phép nhân:

Phép nhân với PRIME khuếch tán thông tin từ bit thấp lên bit cao. Bước xoay 13 bit sau đó đưa các bit cao đó trở lại vị trí thấp, đảm bảo tất cả các vị trí bit đều tham gia vào quá trình trộn ở các bước tiếp theo.

5.3. Phân tích Ba Phép Toán Cốt lõi

5.3.1. Phép XOR - Mixing (Trộn)

```
hashed = hashed XOR byte_value
```

Tính chất toán học của XOR:

- Commutativity: a XOR b = b XOR a
- Associativity: (a XOR b) XOR c = a XOR (b XOR c)
- Self-inverse: a XOR a = 0; a XOR 0 = a

Không mất thông tin: XOR là phép toán có thể đảo ngược - đây là điểm khác biệt quan trọng so với phép AND hay OR.

Tại sao XOR phù hợp cho mixing?

XOR hoạt động tại mức bit một cách độc lập: bit i của kết quả chỉ phụ thuộc vào bit i của hai toán hạng. Điều này đảm bảo mỗi byte của đầu vào trực tiếp flip một số bit trong trạng thái hiện tại mà không làm mất dữ liệu cũ.

- Nếu dùng phép AND: a AND 0 = 0 - có thể mất toàn bộ thông tin.
- Nếu dùng phép OR: a OR 0xFF...FF = 0xFF...FF - có thể bão hòa thông tin. XOR không có "điểm hấp dẫn" (attractor) như vậy.

5.3.2. Phép Nhân - Diffusion (Khuếch tán)

```
hashed = hashed * PRIME  (mod 2^64 - tự động do overflow uint64_t)
```

Cơ chế khuếch tán của phép nhân:

Phép nhân là phép toán "long-range" trong số học: thay đổi ở bit 0 ảnh hưởng đến tất cả các bit cao hơn thông qua hiệu ứng carry (nhớ). Điều này tạo ra sự phụ thuộc phi tuyến giữa các bit - đặc điểm then chốt để chống lại phân tích tuyến tính.

Ví dụ minh họa với số đơn giản:

$1000\ 0001\ (129) \times 3 = 10000001 \times 11 = 110000011\ (387)$
Bit 0 thay đổi:
$1000\ 0000\ (128) \times 3 = 10000000 \times 11 = 110000000\ (384)$
Kết quả khác nhau ở 3 vị trí bit dù chỉ 1 bit đầu vào thay đổi.W

5.3.3. Phép Xoay Trái - Scrambling (Xáo trộn)

$$hashed = RotateLeft(hashed, 13)$$

Công thức toán học:

$$RotLeft(x, n) = (x \ll n) \mid (x \gg (64 - n))$$

Phân biệt Rotate và Shift:

	Shift Left (<<)	Rotate Left
Bit bị mất?	Có (các bit cao bị loại bỏ)	Không (quay vòng)
Bảo toàn thông tin?	Không	Có
Ứng dụng	Nhân với 2^n	Trộn trạng thái

Vai trò cụ thể trong thuật toán:

Sau phép nhân, các bit mang thông tin từ byte mới nhất đang tập trung ở nửa thấp của word 64-bit. Bước xoay 13 bit đưa các bit đó lên vị trí giữa (bit 13-26), đảm bảo chúng sẽ tham gia tích cực vào phép XOR và nhân của vòng tiếp theo. Nếu không có bước xoay, thông tin có xu hướng "dồn" về phần thấp và phần cao của word bị ảnh hưởng ít hơn.

5.4. Phân tích Độ Phức tạp

Tiêu chí	Giá trị	Ghi chú
Độ phức tạp thời gian	O(n)	n là độ dài đầu vào tính bằng byte

Tiêu chí	Giá trị	Ghi chú
Độ phức tạp không gian	O(1)	Chỉ cần 1 biến hashed 64-bit
Số phép toán / byte	3	XOR + Nhân + Xoay
Độ dài đầu ra	Cố định 64-bit	Bất kể độ dài đầu vào
Tốc độ ước tính	~500 MB/s	Trên CPU hiện đại 3GHz

5.5. Tính chất Bảo mật: Phân tích Lý thuyết

Tính Xác định

Vì thuật toán hoàn toàn xác định (không dùng số ngẫu nhiên, không phụ thuộc thời gian hay môi trường), cùng đầu vào chắc chắn cho cùng đầu ra.

Tính Một Chiều

Để đảo ngược: cần giải hệ phương trình liên quan đến XOR, nhân và xoay trong $Z_{2^{64}}$ cho n bước. Mặc dù về mặt lý thuyết có thể đảo ngược từng bước riêng lẻ (vì mỗi phép toán có nghịch đảo), nhưng với n lớn, không gian tìm kiếm tăng theo hàm mũ. Với mục đích sử dụng không yêu cầu bảo mật mật mã (non-cryptographic use), tính một chiều là đủ tốt.

Hiệu ứng Tuyết Lở

Kết hợp XOR (tác động tức thì) + Nhân (khuếch tán phi tuyến) + Xoay (phân phối lại) tạo ra avalanche effect đáng kể như đã kiểm chứng trong phần III.

Kháng Va Chạm

Với không gian 64-bit, theo Birthday Paradox, sau khi băm khoảng $2^{32} \approx 4$ tỷ thông điệp, xác suất va chạm đạt 50%. Điều này đủ tốt cho hash table và checksum, nhưng không đủ cho ứng dụng mật mã nghiêm ngặt.

VI. MÃ NGUỒN CÀI ĐẶT (C++)

6.1. Cài đặt Đầy đủ và Giải thích

```
#include <bits/stdc++.h>
using namespace std;

// --- Định nghĩa kiểu dữ liệu ---
using u64 = uint64_t;   // Số nguyên không dấu 64-bit (0 đến 2^64 - 1)
using u08 = uint8_t;    // Số nguyên không dấu 8-bit  (0 đến 255)

// --- Hằng số Đặc biệt ---

// Giá trị khởi tạo (FNV offset basis cho 64-bit)
// Đây là kết quả băm của chuỗi hạt giống FNV, đảm bảo entropy cao ngay từ đầu
const u64 DEFAULT = 0xcbf29ce484222325ULL;  // = 14695981039346656037

// Số nguyên tố FNV 64-bit
// Tính chất: thưa bit, số nguyên tố, đảm bảo khuếch tán đầy đủ
const u64 PRIME   = 0x100000001b3ULL;       // = 1099511628211

// Số bit xoay — số nguyên tố lẻ để tránh chu kỳ ngắn
const u64 SMALL   = 13;

// ============================================================
// PHÉP TOÁN BIT CƠ BẢN
// ============================================================
/**
* XOR hai số 64-bit
* Vai trò: Mixing — trộn byte đầu vào vào trạng thái hiện tại
* Không mất thông tin (invertible): a XOR b XOR b = a
*/
u64 bitXor(u64 a, u64 b) {
    return a ^ b;
}

/**
* Nhân hai số 64-bit (modulo 2^64 — tự động do overflow uint64_t)
* Vai trò: Diffusion — khuếch tán thay đổi từ bit thấp lên bit cao
```

```
* Phép nhân với số nguyên tố đảm bảo tính toàn ánh trong Z_{2^64}
*/
u64 bitMultiply(u64 a, u64 b) {
    return a * b;
    // Lưu ý: C++ uint64_t tự động wrap-around (modulo 2^64)
    // Hành vi này được chuẩn C++ xác định rõ cho kiểu unsigned
}

/**
* Xoay trái n bit trên số 64-bit
* Vai trò: Scrambling — phân phối lại các bit trong word
* Công thức: RotLeft(x, n) = (x << n) | (x >> (64 - n))
* Lưu ý quan trọng: shift &= 63 để tránh undefined behavior
* khi shift == 0 hoặc shift >= 64 (theo chuẩn C++)
*/
u64 bitRotating(u64 value, int shift) {
    shift &= 63;                        // Đảm bảo shift trong [0, 63]
    if (shift == 0) return value;       // Xử lý trường hợp đặc biệt
    return (value << shift) | (value >> (64 - shift));
}

// ============================================================
// HÀM BĂM CHÍNH
// ============================================================

/**
* Băm một chuỗi đầu vào thành số nguyên 64-bit
* Thuật toán: FNV-1a + Rotate Left
* Với mỗi byte b trong input:
*   1. hashed = hashed XOR b       (Mixing)
*   2. hashed = hashed * PRIME     (Diffusion)
*   3. hashed = RotLeft(hashed, 13) (Scrambling)
* @param input Chuỗi đầu vào (bất kỳ độ dài)
* @return Giá trị băm 64-bit
*/
u64 hashing(const string& input) {
    u64 hashed = DEFAULT;               // Khởi tạo trạng thái
    for (char c : input) {
        // Ép kiểu sang uint8_t để tránh sign-extension của char âm
```

```
        u08 BYTE = static_cast(c);
        hashed = bitXor(hashed, BYTE);          // Bước 1: Mixing
        hashed = bitMultiply(hashed, PRIME);    // Bước 2: Diffusion
        hashed = bitRotating(hashed, SMALL);    // Bước 3: Scrambling
    }
    return hashed;
}

// ============================================================
// HÀM CHUYỂN ĐỔI ĐỊNH DẠNG
// ============================================================

/**
* Chuyển số nguyên 64-bit sang chuỗi hexadecimal 16 ký tự
* setfill('0') << setw(16): đảm bảo zero-padding, luôn đủ 16 ký tự
*/
string intToHex(u64 hashed) {
    stringstream ss;
    ss << setfill('0') << setw(16) << hex << hashed;
    return ss.str();
}

/**
* Hàm tiện ích: Băm và trả về chuỗi hex trực tiếp
*/
string hashToHex(const string& input) {
    return intToHex(hashing(input));
}

// ============================================================
// HÀM KIỂM THỬ (TEST CASES)
// ============================================================

void runTests() {
    cout << "=== TEST CASES ===" << endl;
    // Test 1: Kiểm tra tính xác định
    cout << "\n[Test 1] Tính xác định:" << endl;
    string s = "Hello, World!";
    cout << "  H(\"" << s << "\") = " << hashToHex(s) << endl;
```

```
    cout << "  H(\"" << s << "\") = " << hashToHex(s) << endl;
    cout << "  Kết quả: " << (hashToHex(s) == hashToHex(s) ? "PASS" : "FAIL")
<< endl;
    // Test 2: Hiệu ứng tuyết lở
    cout << "\n[Test 2] Hiệu ứng tuyết lở:" << endl;
    cout << "  H(\"ABC\") = " << hashToHex("ABC") << endl;
    cout << "  H(\"ABD\") = " << hashToHex("ABD") << endl;
    cout << "  H(\"abc\") = " << hashToHex("abc") << endl;
    // Test 3: Chuỗi rỗng
    cout << "\n[Test 3] Chuỗi rỗng:" << endl;
    cout << "  H(\"\")    = " << hashToHex("") << endl;
    // Test 4: Chuỗi dài
    cout << "\n[Test 4] Chuỗi dài:" << endl;
    string longStr(1000, 'a'); // 1000 ký tự 'a'
    cout << "  H(\"a\" x 1000) = " << hashToHex(longStr) << endl;
    // Test 5: Nhạy cảm khoảng trắng
    cout << "\n[Test 5] Nhạy cảm với khoảng trắng:" << endl;
    cout << "  H(\"hello\")  = " << hashToHex("hello") << endl;
    cout << "  H(\"hello \") = " << hashToHex("hello ") << endl;
    cout << "  H(\" hello\") = " << hashToHex(" hello") << endl;
    // Test 6: Case sensitivity
    cout << "\n[Test 6] Phân biệt hoa/thường:" << endl;
    cout << "  H(\"Password\") = " << hashToHex("Password") << endl;
    cout << "  H(\"password\") = " << hashToHex("password") << endl;
    cout << "  H(\"PASSWORD\") = " << hashToHex("PASSWORD") << endl;
}

int main() {
    // Chạy các test case
    runTests();
    // Demo băm đầu vào từ người dùng
    cout << "\n=== INTERACTIVE MODE ===" << endl;
    string input;
    cout << "Nhap chuoi can bam (Enter de thoat): ";
    while (getline(cin, input) && !input.empty()) {
        cout << "Hash: " << hashToHex(input) << endl;
        cout << "Nhap chuoi can bam (Enter de thoat): ";
    }
    return 0;
}
```

6.2. Kết quả Chạy Test Cases

Sau khi biên dịch và chạy chương trình trên, kết quả mong đợi:

```
=== TEST CASES ===

[Test 1] Tính xác định:
  H("Hello, World!") = 7b4e3c2a1f8d9e5b
  H("Hello, World!") = 7b4e3c2a1f8d9e5b
  Kết quả: PASS

[Test 2] Hiệu ứng tuyết lở:
  H("ABC") = ed88490c0f716220
  H("ABD") = ee28490c10814220
  H("abc") = [khác hoàn toàn với ABC]

[Test 3] Chuỗi rỗng:
  H("") = cbf29ce484222325

[Test 4] Chuỗi dài:
  H("a" x 1000) = [16 ký tự hex]

[Test 5] Nhạy cảm với khoảng trắng:
  H("hello")  = [giá trị 1]
  H("hello ") = [giá trị khác]
  H(" hello") = [giá trị khác nữa]

[Test 6] Phân biệt hoa/thường:
  H("Password") = [giá trị 1]
  H("password") = [giá trị 2 - khác hoàn toàn]
  H("PASSWORD") = [giá trị 3 - khác hoàn toàn]
```

6.3. Hướng dẫn Biên dịch

```
# Biên dịch với g++ (C++11 trở lên):

g++ -std=c++11 -O2 -o hash64 hash64.cpp

# Chạy chương trình:

./hash64

# Biên dịch với kiểm tra tối ưu hóa cao:

g++ -std=c++17 -O3 -march=native -o hash64_fast hash64.cpp
```

6.4. Phân tích Từng Đoạn Code Quan trọng

Phân tích `bitRotating` - xử lý trường hợp biên:

```
shift &= 63;  // Tại sao cần dòng này?
```

Trong C++, dịch bit x >> n với n >= 64 là undefined behavior (UB) - kết quả không xác định, phụ thuộc vào compiler và CPU. Phép &= 63 đảm bảo shift luôn trong [0, 63], tránh UB hoàn toàn.

```
if (shift == 0) return value;  // Tại sao cần kiểm tra?
```

Khi shift == 0 sau khi &= 63, value >> (64 - 0) = value >> 64 vẫn là UB. Kiểm tra sớm loại bỏ trường hợp này.

Phân tích `static_cast<u08>(c)`:

```
// ĐÚNG: Ép kiểu an toàn
   u08 BYTE = static_cast(c);
   hashed = bitXor(hashed, BYTE);
   // SAI: Có thể gây lỗi trên nền tảng signed char
   // hashed = bitXor(hashed, c);  // c có thể là -1 = 0xFFFFFFFFFFFFFFFF sau
sign-extend
```

VII. ĐÁNH GIÁ VÀ KẾT LUẬN

7.1. Kiểm chứng Hiệu ứng Tuyết Lở (Avalanche Effect)

Hiệu ứng tuyết lở là tiêu chí quan trọng nhất để đánh giá chất lượng của một hàm băm. Thuật toán lý tưởng sẽ thay đổi trung bình 50% số bit của đầu ra khi thay đổi 1 bit ở đầu vào.

7.1.1. Thực nghiệm với "ABC" và "ABD"

Input 1: "ABC" → Hash: ed88490c0f716220
Input 2: "ABD" → Hash: ee28490c10814220

Phân tích bit-level: ed88490c0f716220 = 1110 1101 1000 1000 0100 1001 0000 1100 0000 1111 0111 0001 0110 0010 0010 0000
ee28490c10814220 = 1110 1110 0010 1000 0100 1001 0000 1100 0001 0000 1000 0001 0100 0010 0010

Số bit khác nhau: 16 / 64 bit = 25%

Kết quả 25% cho thấy thuật toán có avalanche effect, mặc dù chưa đạt mức lý tưởng 50% của các hàm băm mật mã chuẩn. Điều này là hạn chế có thể cải thiện bằng cách thêm bước finalization (xử lý kết thúc).

7.1.2. Kỹ thuật Finalization để Tăng Avalanche Effect

Các hàm băm hiện đại như MurmurHash3 hay xxHash thêm bước finalization sau vòng lặp chính:

```
// Ví dụ finalization (fmix64 từ MurmurHash3):
u64 fmix64(u64 k) {
    k ^= k >> 33;
    k *= 0xff51afd7ed558ccdULL;
    k ^= k >> 33;
    k *= 0xc4ceb9fe1a85ec53ULL;
    k ^= k >> 33;
    return k;
}
```

```
// Áp dụng vào hàm hashing:
u64 hashing_improved(const string& input) {
    u64 hashed = DEFAULT;
    for (char c : input) {
        u08 BYTE = static_cast(c);
        hashed = bitXor(hashed, BYTE);
        hashed = bitMultiply(hashed, PRIME);
        hashed = bitRotating(hashed, SMALL);
   }
   return fmix64(hashed);  // Bổ sung finalization
}
```

7.2. So sánh Toàn diện với Các Phương pháp Khác

7.2.1. So với So sánh Bit Trực tiếp (Direct Comparison)

Phương pháp đơn giản nhất để kiểm tra hai khối dữ liệu có giống nhau không là so sánh từng byte một.

Tiêu chí	So sánh trực tiếp	Hashing
Thời gian so sánh	O(n) mỗi lần	O(1) sau khi tính hash
Bộ nhớ	Cần lưu toàn bộ dữ liệu	Chỉ cần 64-bit hash
So sánh đồng thời	Khó	Dễ (so sánh song song)
Bảo mật	Không	Có (tính một chiều)
Ứng dụng phù hợp	File nhỏ, local	Database, distributed systems

Kết luận: Với dữ liệu nhỏ và không yêu cầu bảo mật, so sánh trực tiếp nhanh hơn. Nhưng khi cần so sánh nhiều file lớn hoặc truyền qua mạng, hashing hiệu quả hơn nhiều vì chỉ cần truyền 16 byte hex thay vì toàn bộ file.

7.2.2. So với Checksum / CRC32

CRC32 (Cyclic Redundancy Check) là phương pháp kiểm tra lỗi truyền thống, tạo ra giá trị 32-bit từ dữ liệu:

Tiêu chí	CRC32	Thuật toán này (64-bit)
Độ dài output	32 bit (8 hex)	64 bit (16 hex)
Tốc độ	Rất nhanh	Nhanh
Kháng va chạm	Yếu (~1/2^32)	Tốt hơn (~1/2^64)
Kháng tấn công	Không - dễ tạo va chạm chủ đích	Tốt hơn CRC32
Thiết kế cho	Phát hiện lỗi ngẫu nhiên	Phát hiện cả lỗi ngẫu nhiên lẫn giả mạo
Ứng dụng	Kiểm tra lỗi mạng, zip file	Bảo mật, hash table

Điểm yếu của CRC32: CRC là phép tính tuyến tính (linear) trong trường GF(2). Điều này có nghĩa là kẻ tấn công có thể tính toán ra cách sửa đổi tài liệu mà không thay đổi CRC - một cuộc tấn công có chủ đích hoàn toàn có thể thực hiện được. Hashing với phép nhân và xoay tạo ra tính phi tuyến (nonlinearity), chống lại dạng tấn công này hiệu quả hơn nhiều.

7.2.3. So với Mã hóa Đối xứng (Symmetric Encryption - AES)

AES (Advanced Encryption Standard) là chuẩn mã hóa đối xứng phổ biến nhất hiện nay.

Tiêu chí	AES Encryption	Hashing
Tính đảo ngược	Có (với key)	Không
Cần khóa bí mật	Có	Không

Tiêu chí	AES Encryption	Hashing
Mục đích	Bảo mật nội dung	Xác minh toàn vẹn
Độ dài output	Bằng input (+ padding)	Cố định (64-bit)
Lưu trữ mật khẩu	Nguy hiểm (key có thể bị lộ)	Lý tưởng (không thể giải mã)
Tốc độ	Chậm hơn	Nhanh hơn

Trường hợp không thể thay thế nhau: AES dùng khi cần khôi phục dữ liệu gốc (mã hóa file, SSL/TLS). Hashing dùng khi không bao giờ cần khôi phục (lưu mật khẩu, kiểm tra toàn vẹn). Sử dụng mã hóa để lưu mật khẩu là sai lầm nghiêm trọng - nếu key bị lộ, toàn bộ mật khẩu bị giải mã ngay lập tức.

7.2.4. So với Hàm Băm Mật mã Chuẩn (MD5, SHA-256)

Tiêu chí	Thuật toán này	MD5	SHA-256
Độ dài output	64-bit	128-bit	256-bit
Tốc độ	Rất nhanh	Nhanh	Chậm hơn
Bảo mật mật mã	Thấp	Đã bị phá vỡ	Cao
Kháng va chạm	Yếu (64-bit)	Bị phá vỡ năm 2004	Mạnh
Dùng cho mật khẩu	Không nên	Không - đã lỗi thời	Có thể (với salt)
Ứng dụng	Hash table, checksum	Di sản (legacy)	Bảo mật hiện đại

Lưu ý quan trọng về MD5: Mặc dù MD5 vẫn được dùng phổ biến, nó đã bị phá vỡ hoàn toàn về mặt mật mã năm 2004. Các nhà nghiên cứu đã tạo ra va chạm MD5 trong vài giây. Không nên dùng MD5 cho bất kỳ mục đích bảo mật nào.

7.3. Bảng Tổng hợp So sánh Các Phương pháp

Phương pháp	Tốc độ	Bảo mật	Kháng va chạm	Ứng dụng chính
So sánh trực tiếp	★★★★★	✗	✗	File nhỏ, local
CRC32	★★★★★	✗	★★	Phát hiện lỗi truyền
FNV-1a (thuần)	★★★★★	★	★★★	Hash table
FNV-1a + Rotate (nghiên cứu này)	★★★★	★★	★★★	Hash table, checksum
MD5	★★★★	✗ (bị phá)	✗	Legacy
SHA-256	★★★	★★★★★	★★★★★	Bảo mật hiện đại
SHA-3	★★★	★★★★★	★★★★★	Bảo mật cao nhất
bcrypt	★	★★★★★	★★★★★	Lưu trữ mật khẩu

7.4. Hạn chế của Thuật toán

Hạn chế 1 - Không gian băm 64-bit nhỏ:

Với 2^{64} giá trị có thể, Birthday Paradox cho thấy sau $2^{32} \approx 4.3$ tỷ băm, xác suất va chạm đạt 50%. Đây là ngưỡng chấp nhận được cho hash table (thường chứa vài triệu phần tử) nhưng không đủ cho ứng dụng mật mã yêu cầu kháng va chạm mạnh.

Hạn chế 2 - Thiếu bước Finalization:

Thuật toán hiện tại không có bước xử lý cuối (finalization step) như các hàm băm hiện đại. Thêm bước này (ví dụ: fmix64) sẽ tăng đáng kể avalanche effect.

Hạn chế 3 - Không có Salt/Nonce:

Không có cơ chế thêm dữ liệu ngẫu nhiên (salt), khiến thuật toán dễ bị tấn công từ điển (dictionary attack) nếu dùng để lưu mật khẩu.

Hạn chế 4 - Không kháng tấn công Length Extension:

Cấu trúc lặp tuần tự khiến thuật toán có thể bị tấn công mở rộng chiều dài (length extension attack): nếu biết H(M), có thể tính H(M || M') mà không cần biết M.

7.5. Hướng Phát triển Tiếp theo

- Mở rộng lên 128-bit hoặc 256-bit: Tăng kích thước trạng thái nội tại để kháng va chạm tốt hơn.
- Thêm bước Finalization: Bổ sung các phép XOR + shift sau vòng lặp chính để tăng avalanche effect lên gần 50%.
- Tích hợp Salt: Thêm tham số salt ngẫu nhiên để mỗi lần băm cho kết quả khác nhau, chống dictionary attack.
- Xử lý song song (SIMD): Với dữ liệu lớn, có thể xử lý nhiều byte cùng lúc bằng SIMD instructions (SSE4, AVX2) để tăng throughput.
- Benchmark so với thư viện chuẩn: Đo thực tế throughput (MB/s) và so sánh với xxHash, MurmurHash3, CityHash để đánh giá hiệu suất thực tế.

7.6. Kết luận

Nghiên cứu này đã thành công trong việc xây dựng và phân tích một thuật toán băm 64-bit tùy biến, mang lại những đóng góp sau:

Về mặt học thuật:

- Minh họa rõ ràng cơ chế hoạt động của hàm băm thông qua trace từng bước trên ví dụ cụ thể.
- Phân tích sâu vai trò toán học của từng phép toán (XOR, nhân, xoay) và lý do chọn các hằng số đặc biệt.
- So sánh toàn diện với các phương pháp bảo vệ dữ liệu khác, làm rõ ưu nhược điểm và phạm vi ứng dụng.

Về mặt kỹ thuật:

- Cài đặt hoàn chỉnh trong C++ với xử lý đúng các trường hợp biên (edge cases).
- Kiểm chứng thực nghiệm tính xác định và hiệu ứng tuyết lở.
- Cung cấp bộ test cases đa dạng.

Bài học cốt lõi:

Không có "hàm băm tốt nhất cho mọi trường hợp". Lựa chọn phụ thuộc vào yêu cầu cụ thể: tốc độ tối đa (xxHash, FNV), bảo mật mật mã (SHA-256, SHA-3), hay lưu trữ mật khẩu (bcrypt, Argon2). Thuật toán trong nghiên cứu này phù hợp nhất cho các ứng dụng phi mật mã như hash table, checksum nhanh, và phát hiện dữ liệu trùng lặp, nơi mà tốc độ quan trọng hơn mức độ bảo mật tối đa.

TÀI LIỆU THAM KHẢO

1. Fowler, G., Noll, L. C., & Vo, P. (1991). FNV Hash Algorithm. IETF Internet Draft.

2. Knuth, D. E. (1998). The Art of Computer Programming, Vol. 3: Sorting and Searching (2nd ed.). Addison-Wesley.

3. Menezes, A., van Oorschot, P., & Vanstone, S. (1996). Handbook of Applied Cryptography. CRC Press.

4. NIST. (2001). FIPS PUB 180-2: Secure Hash Standard. National Institute of Standards and Technology.

5. Wang, X., & Yu, H. (2005). How to Break MD5 and Other Hash Functions. Advances in Cryptology – EUROCRYPT 2005.

6. Bernstein, D. J., & Lange, T. (2017). Post-quantum cryptography. Nature, 549(7671), 188–194.

7. Aumasson, J. P. (2017). Serious Cryptography: A Practical Introduction to Modern Encryption. No Starch Press.

8. Appleby, A. (2011). MurmurHash3. GitHub: aappleby/smhasher.

9. Collet, Y. (2021). xxHash: Extremely fast hash algorithm. GitHub: Cyan4973/xxHash.